

TRIBHUVAN UNIVERSITY

FACULTY OF HUMANITIES AND SOCIAL SCIENCES

A Project Report On

"Bloodbridge : Blood Donation management system"

Submitted to:

Department of Computer Application

Xavier International College

In partial fulfillment of the requirements for the bachelors in computer application.

Submitted by:

Lakhmaya Tamang

BCA 8th semester

University SN: 6-2-546-27-2021

Pemba Chhumu Sherpa

BCA 8th semester

University SN: 6-2-546-30-2021

Under the supervision of

Ankit Nepal



Tribhuvan University

Faculty of Humanities and Social Sciences

Xavier International College

Boudha, Kathmandu

SUPERVISOR'S RECOMMENDATION

I hereby recommend that this project prepared under my supervision by **Lakhmaya Tamang** and **Pemba Chhumu Sherpa** entitled **"Bloodbridge - Align your blood needs"** in partial fulfillment of the requirements for the degree of Bachelor of Computer Application is recommended for the final evaluation.

_____

SIGNATURE

Ankit Nepal

SUPERVISOR

Humanities and Social Sciences

Bachelor in Computer Application

Xavier International College, Boudha, Kathmandu

Tribhuvan University

Faculty of Humanities and Social Sciences

Xavier International College

Boudha, Kathmandu

LETTER OF APPROVAL

This is to certify that this project prepared by **Lakhmaya Tamang** and **Pemba Chhumu Sherpa** entitled **"Bloodbridge - Align your blood needs"** in partial fulfillment of the requirements for the degree of Bachelor in Computer Application has been evaluated. In our opinion it is satisfactory in the scope and quality as a project for the required degree

_____ Ankit Nepal Supervisor Bachelor in Computer Application Xavier International College Boudha, Kathmandu	_____ Rajesh Nepal Co-ordinator Bachelor in Computer Application Xavier International College Boudha, Kathmandu
_____ Internal Examiner	_____ External Examiner

ABSTRACT

The Blood Donation Management System is a web-based platform developed using the Laravel PHP framework to streamline the process of connecting blood donors with recipients in need. The system allows users to register as donors, maintain a complete donation history, and manage their availability status. Recipients can submit blood requests by specifying the required blood group, quantity, urgency, and location. Administrators oversee the entire platform, verifying donor information, managing user accounts, and monitoring requests to ensure the authenticity and reliability of the data.

The application prioritizes usability through a clean and responsive interface, and it provides real-time donor-recipient matching based on blood group compatibility, geographic location, and donor eligibility. Secure data handling is ensured through authentication, role-based access control, and input validation, which protect sensitive personal and medical information. By leveraging modern web technologies for social good, the project aims to reduce the time needed to find suitable donors and to promote timely and efficient blood donation, particularly in emergency situations.

Keywords: Blood Donation, Donor Management, Laravel, PHP, Web Application, Real-Time Matching, Blood Request, Role-Based Access Control, Emergency Response, Data Security

ACKNOWLEDGEMENT

We would like to express our heartfelt gratitude to all those who supported and guided us throughout the development of this project.

First and foremost, we are deeply thankful to our mentor, **Mr. Ankit Nepal**, for his invaluable guidance, constant support, and insightful feedback, which greatly shaped the direction and quality of this work.

We extend our sincere thanks to the Head of Department and the College Coordinator for their continuous encouragement and for providing the necessary framework to successfully carry out this project.

We would also like to acknowledge our seniors for their helpful suggestions and guidance at different stages of the project.

A special thanks to our brother Pradeep Dhungana for his unwavering motivation and belief in our efforts, and to our friend, Suprav Kandel, for his consistent assistance, collaboration, and technical input throughout the development process.

Lastly, we would like to express our appreciation to our college, Xavier International College, for providing a supportive academic environment and essential resources that enabled the successful completion of this project.

This project would not have been possible without the collective support and inspiration from everyone mentioned above.

Thank you.
Yours Sincerely,

Lakhmaya Tamang
Pemba Chhumu Sherpa

Table of Contents

ABSTRACT ..... **i**

ACKNOWLEDGEMENT ..... **ii**

CHAPTER 1. INTRODUCTION **4**

1.1 Introduction 4

1.2 Problem Statement 4

1.3 Objectives 5

1.3.1 General Objective: 5

1.3.2 Specific Objectives: 5

1.4 Scope and Limitations 6

1.5 Development Methodology 6

1.5.1 Requirement Analysis 7

1.5.2 System Design 7

1.5.3 Technology Stack 7

1.5.4 Development 7

1.5.5 Testing 7

1.5.6 Deployment 7

1.6 Report Organization 8

CHAPTER 2. BACKGROUND STUDY AND LITERATURE REVIEW **10**

2.1 Background Study 10

2.2 Literature Review 11

CHAPTER 3. SYSTEM ANALYSIS AND DESIGN **12**

3.1 System Analysis 12

3.1.1 Requirement Analysis 12

3.1.2 Feasibility Analysis 13

3.1.3 Object Modeling using Class and Object Diagrams 14
3.1.4 Dynamic Modeling using State and Sequence Diagram 16
3.1.5 Process Modeling using Activity Diagram 18
3.2 System Design 19
3.2.1 Refinement of Class, Object, State, Sequence, and Activity Diagrams 20
3.2.2 Component Diagram 20
3.2.3 Deployment Diagram 21
3.3 Algorithm Details 22
CHAPTER 4. IMPLEMENTATION AND TESTING 24
4.1 Implementation 24
4.1.1 Tools Used 24
4.1.2 Implementation Details of Modules 25
4.2 Testing 27
4.2.1 Test Cases for Unit Testing 27
4.2.2 Test Cases for System Testing 29
4.3Result Analysis 32
CHAPTER 5. CONCLUSION AND FUTURE RECOMMENDATIONS 34
5.1 Conclusion 34
5.2 Lessons Learned / Outcome 34
5.3 Future Recommendations 34
References 36
Chapter 6. Appendices 37

LIST OF ABBREVIATIONS

Abbreviation	Full Form
AI	Artificial Intelligence
API	Application Programming Interface
CRUD	Create, Read, Update, Delete
CSRF	Cross-Site Request Forgery
CSS	Cascading Style Sheets
CSV	Comma Separated Values
DB	Database
DFD	Data Flow Diagram
ER Diagram	Entity Relationship Diagram
HTML	Hyper Text Markup Language
JS	JavaScript
MVC	Model-View-Controller
ORM	Object-Relational Mapping
PHP	PHP: Hypertext Preprocessor
REPL	Read-Eval-Print Loop
SDLC	Software Development Life Cycle
SMS	Short Message Service
SQL	Structured Query Language
UI	User Interface
UX	User Experience
VS Code	Visual Studio Code
XSS	Cross-Site Scripting

LIST OF FIGURES

Figure1.1 Software Development Life Cycle of Bloodbridge.....8

Figure 3.1: Use case Diagram of Bloodbridge.....13

Figure 3.2: **Class Diagram of Bloodbridge** 15

Figure 3.3: Object Diagram of Bloodbridge.....16

Figure 3.4: State Diagram of Bloodbridge.....17

Figure 3.5: Sequence Diagram of Bloodbridge 18

Figure 3.6:Activity diagram of Bloodbridge 19

Figure 3.7: Component Diagram of bloodbridge.....21

Figure 3.8: Deployment diagram of Bloodbridge 21

LIST OF TABLES

Table 4.1: Unit testing of User and authentication Test in Bloodbridge 27

Table 4.2: Unit testing of Donor Profile Test in Bloodbridge.....28

Table 4.3: Unit Testing of Blood Request Test in Blood bridge 28

Table 4.4: Unit Testing of Donor Matching Test in Bloodbridge.....29

Table 4.5:System Testing of Test Environment in BloodBridge 30

CHAPTER 1. INTRODUCTION

1.1 Introduction

BloodBridge is a web-based blood donation management system developed to connect blood recipients with eligible donors efficiently. The platform simplifies donor discovery, registration, and management through a user-friendly interface and a robust backend built with Laravel PHP. BloodBridge is a digital solution designed to enhance the accessibility and organization of blood donation services. Built using Laravel PHP with native Blade syntax, the system enables users in need of blood to quickly access a list of eligible and nearby donors. It maintains donor eligibility records (based on a 3-month donation cycle), allows new donor registrations, and provides an admin panel for manually entering offline donor data.

The platform addresses several shortcomings of existing systems, such as lack of mobile optimization, limited administrative capabilities, and missing features like automatic eligibility filtering. BloodBridge ensures that both users and administrators can interact with the system efficiently, whether for emergency needs, donor management, or tracking donation history. The system's design follows modern web development practices with mobile responsiveness, a clean UI, and a scalable backend.

By integrating important real-time features and simplifying the process of blood donation management, BloodBridge contributes to making voluntary blood donation more accessible, organized, and community-driven.

1.2 Problem Statement

Despite the growing need for blood in medical emergencies, many regions still lack a streamlined platform to connect willing donors with those in urgent need. Traditional methods—such as social media posts, personal calls, or scattered donor records—are

inefficient, unorganized, and time-consuming. There is a clear need for a centralized system that simplifies donor-recipient interaction, filters eligible donors, and maintains an accurate donation history.

Key Problems Identified:

- Lack of a centralized system for accessing eligible blood donors.
- Difficulty in manually tracking donor eligibility (3-month donation rule).
- Inability to quickly connect recipients with donors in the same geographic region.
- Limited administrative control for managing offline donors and correcting data.
- Existing platforms either lack admin features, mobile responsiveness, or eligibilityvalidation.
- Inefficient record-keeping and absence of donor engagement mechanisms.

1.3 Objectives

1.3.1 General Objective:

To develop an efficient and user-friendly web-based platform named BloodBridge that connects blood donors with recipients, ensuring quick access to eligible donors based on location and donation history.

1.3.2 Specific Objectives:

- To enable users to request blood donor information filtered by eligibility and geographic location.
- To allow donors to register, manage their profiles, and view their donation history.
- To notify donors when they become eligible to donate again (after a 3-month interval).
- To provide an admin dashboard for managing users and manually entering donor records.
- To maintain a secure and organized donor database for accurate tracking and retrieval.
- To design a mobile-responsive interface for seamless access across devices.

- To implement search and filtering features based on blood type, location, and availability.

1.4 Scope and Limitations

Scope:

- The system focuses on streamlining the process of connecting blood donors and recipients through a centralised platform.
- Donors can register, update their profile, and track their donation history.
- Recipients can request donor information filtered by location and eligibility (i.e., at least 3 months since last donation).
- The admin panel provides functionality for manual donor entry and user management.
- Eligibility is calculated automatically based on donation date and current date.
- The system is developed using Laravel PHP with native Blade templates, ensuring clean backend logic and responsive frontend design.

Limitations:

- The platform does not include real-time communication or automatic call/SMS functionality (requires third-party services).
- Medical eligibility factors (other than the 3-month interval) are not deeply validated.
- The system does not integrate with hospital or blood bank databases.
- Offline access to the platform is not supported.
- Notification/reminder system is limited to basic in-system alerts or emails (if configured).

1.5 Development Methodology

The development of the **BloodBridge** system followed the **Software Development Life Cycle (SDLC)** using an **Iterative and Incremental Model**. This approach allowed for progressive refinement through feedback and testing after each development phase. The key phases followed are:

1.5.1 Requirement Analysis

I. Collected functional and non-functional requirements through study of existing systems and user expectations.
II. Identified major user roles (admin, donor, requester) and their interactions with the system.

1.5.2 System Design

I. Created high-level architecture and database schema.
II. Designed ER diagrams, DFDs, and interface wireframes to visualize data flow and interaction.

1.5.3 Technology Stack

I. Backend: Laravel (PHP framework)
II. Frontend: Blade templating engine with HTML, CSS (Bootstrap), and JavaScript
III. Database: MySQL
IV. Tools: VS Code, Git, phpMyAdmin, Lucidchart

1.5.4 Development

I. Followed modular implementation starting from authentication, then donor/request management, and finally the admin panel.
II. Each module was tested incrementally before integrating into the core system.

1.5.5 Testing

I. Performed unit testing for individual modules and system testing for overall flow.
II. Manual and automated test cases were used to validate form submissions, user eligibility logic, and admin operations.

1.5.6 Deployment

I. Deployed the project on a local server (XAMPP) for testing.
II. Later stages will support deployment on a live hosting environment for public use.

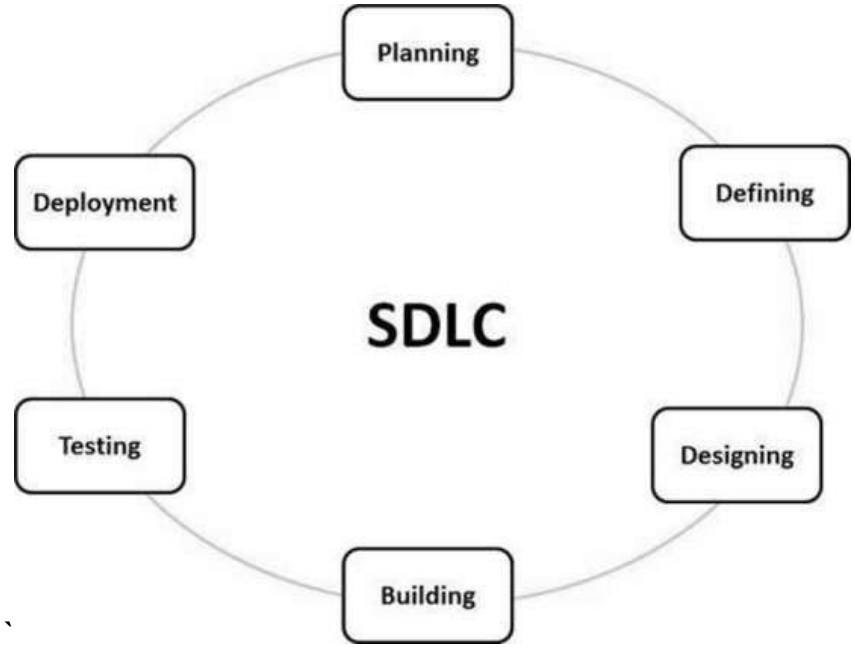


Figure1.1 Software Development Life Cycle of Bloodbridge

1.6 Report Organization

Chapter 1: Introduction

Chapter one introduces the concept of this project. It describes the problems that exist and how its objective can tackle it. It also presents the scope and limitations of the project.

Chapter 2: Background study and literature review

This chapter focuses on the basic ideology of how this project will be built. It traces out the study of different platforms and their workings.

Chapter 3: System analysis and design

This chapter describes the requirements gathering, feasibility assessment, and designing of the project. It includes diagrams, functionality analysis, requirement gathering technique and process model.

Chapter 4: Implementation and testing

This chapter is designed to give information about how the project has been implemented, what kind of software and tools has been used and the type of testing that the project has gone through.

Chapter 5: Conclusion and future recommendations

This chapter includes the possible outcome of this project, conclusion and future recommendations.

CHAPTER 2. BACKGROUND STUDY AND LITERATURE REVIEW

In the modern age, efficient management of blood donations is essential for saving lives. Traditional systems of connecting donors and recipients are often slow, fragmented, and heavily reliant on manual communication. With the increasing demand for blood during emergencies and medical procedures, there's a growing need for a centralized platform that can quickly match eligible donors to those in need based on location, blood group, and donation history.

BloodBridge is designed to address these challenges by digitizing and automating the blood donation management process. It connects donors, recipients, and administrators through a unified interface while ensuring that only eligible donors (those who haven't donated in the past 3 months) are listed. The system also simplifies record-keeping for both public users and administrators.

2.1 Background Study

Blood donation has long been recognized as a critical component of healthcare infrastructure. In Nepal and many other countries, public awareness around donation exists, but access to donor data and real-time communication remains a bottleneck. Various campaigns and local initiatives have attempted to maintain donor lists using spreadsheets or social platforms like Facebook, but these are neither sustainable nor secure.

Moreover, traditional blood banks and hospitals often maintain physical records, which are prone to errors and delays. The lack of an integrated digital system results in slower response times, especially during emergencies.

Digital platforms like BloodBridge offer a practical solution to bridge these gaps. By collecting donor data, tracking donation history, and offering location-based search, it creates a more responsive and reliable network. This background motivated the development of BloodBridge to modernize and simplify the entire process of blood donor management using web-based technology.

2.2 Literature Review

Blood donation management systems have been widely developed to improve the efficiency of blood banks and facilitate quick matching between donors and recipients. Existing platforms typically offer donor registration, eligibility verification, and location-based search functionalities.

For example, **Red Cross Blood Services** (https://www.redcrossblood.org/) is one of the largest and most well-known blood donation organizations worldwide. Their platform allows donors to register, schedule appointments, and track donation history. It also provides information on donor eligibility and location-based blood drives.

Another example is **Blood Donors India** (https://www.blooddonors.in/), a platform that connects donors and recipients across India. It supports blood group and location-based donor search, and maintains records of donor eligibility based on donation intervals.

Blood4Life (https://www.blood4life.in/) is a mobile app-based system that not only manages donor information but also sends reminders to donors when they become eligible to donate again, encouraging regular donations.

These systems emphasize usability, real-time data, and automated eligibility checks, but some still rely heavily on manual data entry and lack comprehensive donor history tracking. Additionally, many platforms do not provide detailed administrative tools to manage donors manually or analyze donation trends, which are important for maintaining database accuracy and improving blood availability.

This project builds upon these concepts by integrating donor eligibility checks (3-month interval), location-based search, manual donor management via an admin panel, and comprehensive tracking of donor donation history, aiming to create a more intuitive and functional blood donation management system.

CHAPTER 3. SYSTEM ANALYSIS AND DESIGN

This chapter presents the detailed analysis and design of the Blood Donation Management System. It covers the understanding of system requirements, feasibility considerations, and the design models including data and process representations. The aim is to ensure the system is well-planned to meet user needs effectively and efficiently.

3.1 System Analysis

System analysis focuses on gathering and studying the requirements of the system from the perspective of different stakeholders such as donors, recipients, and administrators. This phase identifies the functionalities that the system must provide and defines the scope of operations to ensure smooth and reliable blood donation management.

3.1.1 Requirement Analysis

The requirement analysis phase involves collecting and defining the functional and non-functional requirements of the Blood Donation Management System. This process ensures that the system meets the expectations of all users and stakeholders.

Functional Requirements:

- Donors can register by providing personal details, blood group, location, and donation history.
- Donors' eligibility is automatically checked based on a minimum 3-month interval since their last donation.
- Users in need of blood can request information about eligible donors filtered by location and blood group.
- An admin panel allows manual entry and management of donor information.
- The system keeps track of the number of donations made by each donor.
- Donors can update their profiles and donation records.
- Notifications or alerts (optional/future) can remind donors when they become eligible to donate again.

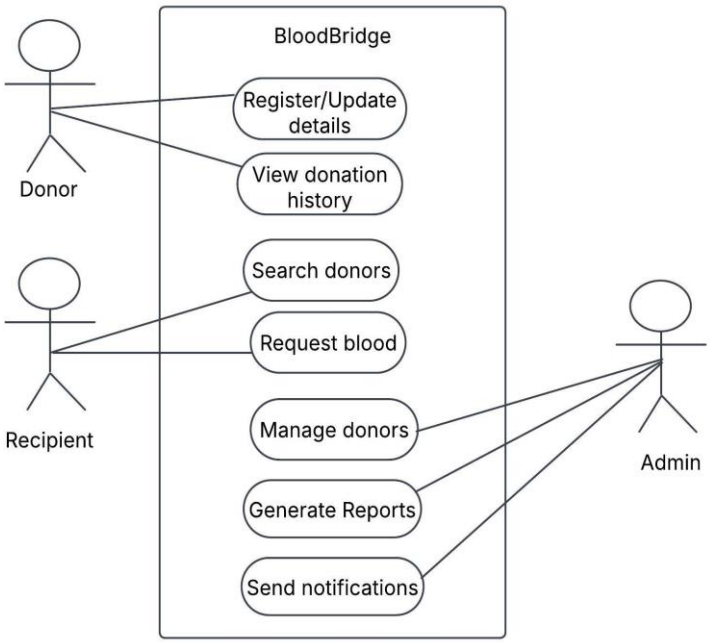


Figure 3.1: Use case Diagram of Bloodbridge

Non-Functional Requirements:

- The system should be user-friendly with an intuitive interface.
- Data security and privacy of donor information must be ensured.
- The system should be scalable to support increasing numbers of donors and requests.
- The platform should be accessible via web browsers on various devices.
- Response time for search and requests should be minimal to ensure timely access to donor information.

3.1.2 Feasibility Analysis

Before development, a feasibility study was conducted to assess the practicality and viability of the Blood Donation Management System. The analysis covers technical, economic, operational, and schedule feasibility aspects.

Technical Feasibility:

The project uses widely supported technologies such as PHP and MySQL, which are readily available and well-understood by the development team. The required infrastructure, including web hosting and database servers, is accessible and compatible with the project requirements. No significant technical barriers are anticipated.

Economic Feasibility:

The project is cost-effective, leveraging open-source tools and frameworks, which reduces development and deployment costs. The system aims to automate blood donation management, potentially reducing manual labor and improving efficiency, thus delivering value to stakeholders.

Operational Feasibility:

The system is designed to be user-friendly for donors, recipients, and administrators. Training requirements are minimal due to the intuitive interface. The platform will improve blood donation processes, making it more likely to be accepted and used by the target audience.

Schedule Feasibility:

The project timeline is realistic, with development, testing, and deployment planned within a defined period. The team has adequate experience and resources to meet deadlines without compromising quality.

3.1.3 Object Modeling using Class and Object Diagrams

Class Diagram

The class diagram for Bloodbridge represents the system's static structure. It helps in understanding the system's logical design by showing the main classes, their attributes, and the relationships that connect them.

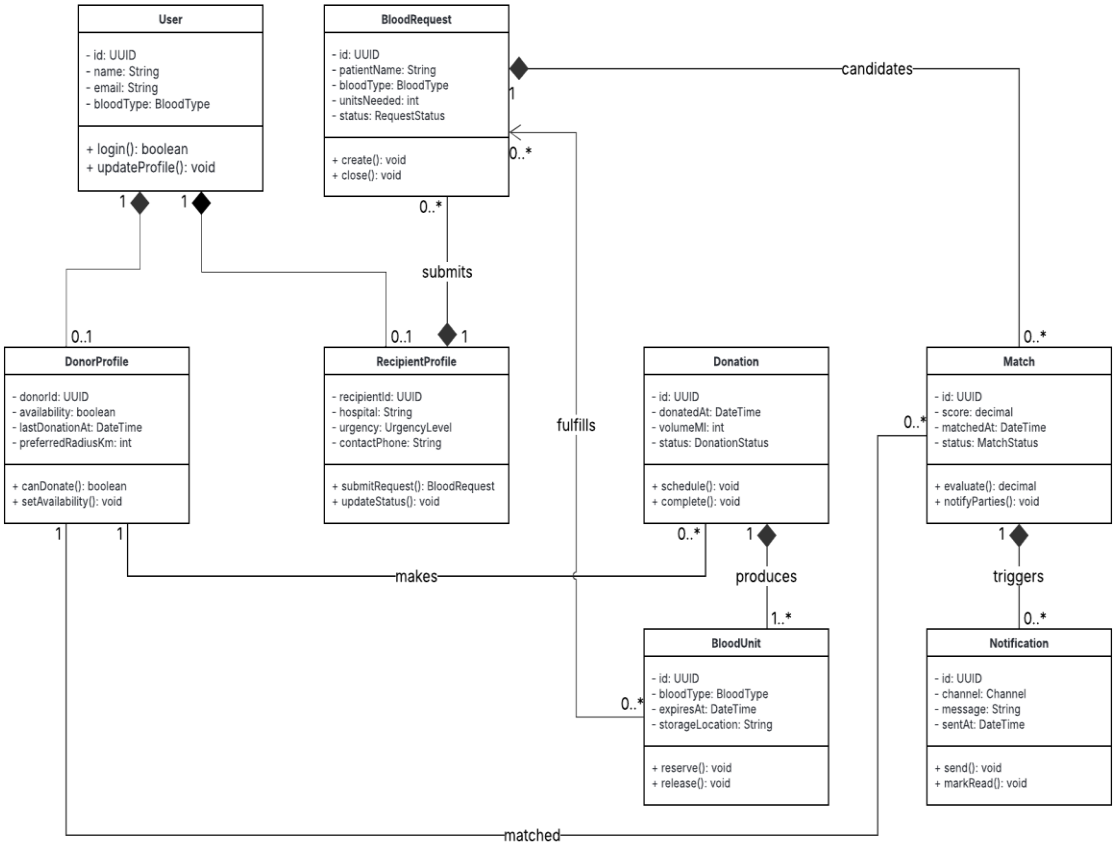


Figure 2.2: Class Diagram of Bloodbridge

Object Diagram

The object diagram for Bloodbridge represents the system's dynamic structure. It illustrates specific instances of classes at a particular moment in time, showing real-world data values for attributes and the relationship between objects.

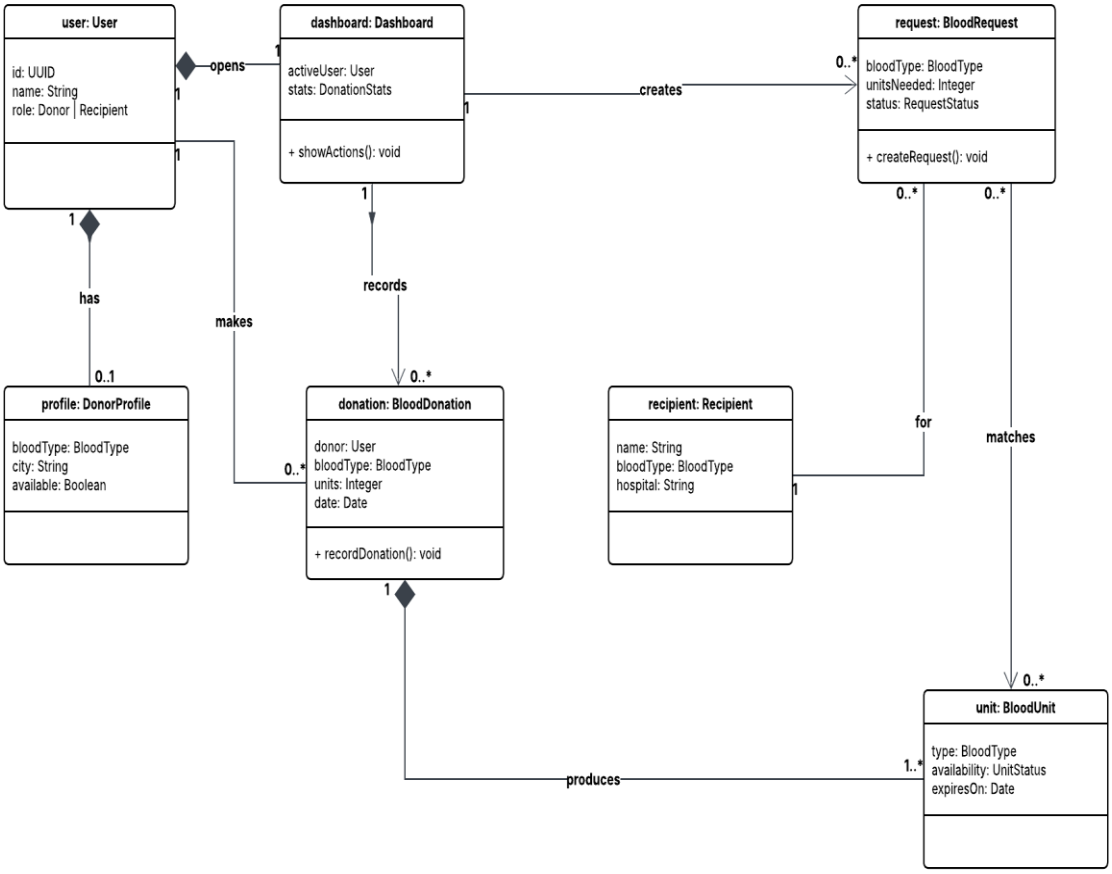


Figure 3.3: Object Diagram of Bloodbridge

3.1.4 Dynamic Modeling using State and Sequence Diagram

State Diagram

The state diagram for Bloodbridge represents the lifecycle of objects within the system, showing the various states a donor listing or user report can occupy and the transitions triggered by events or actions.

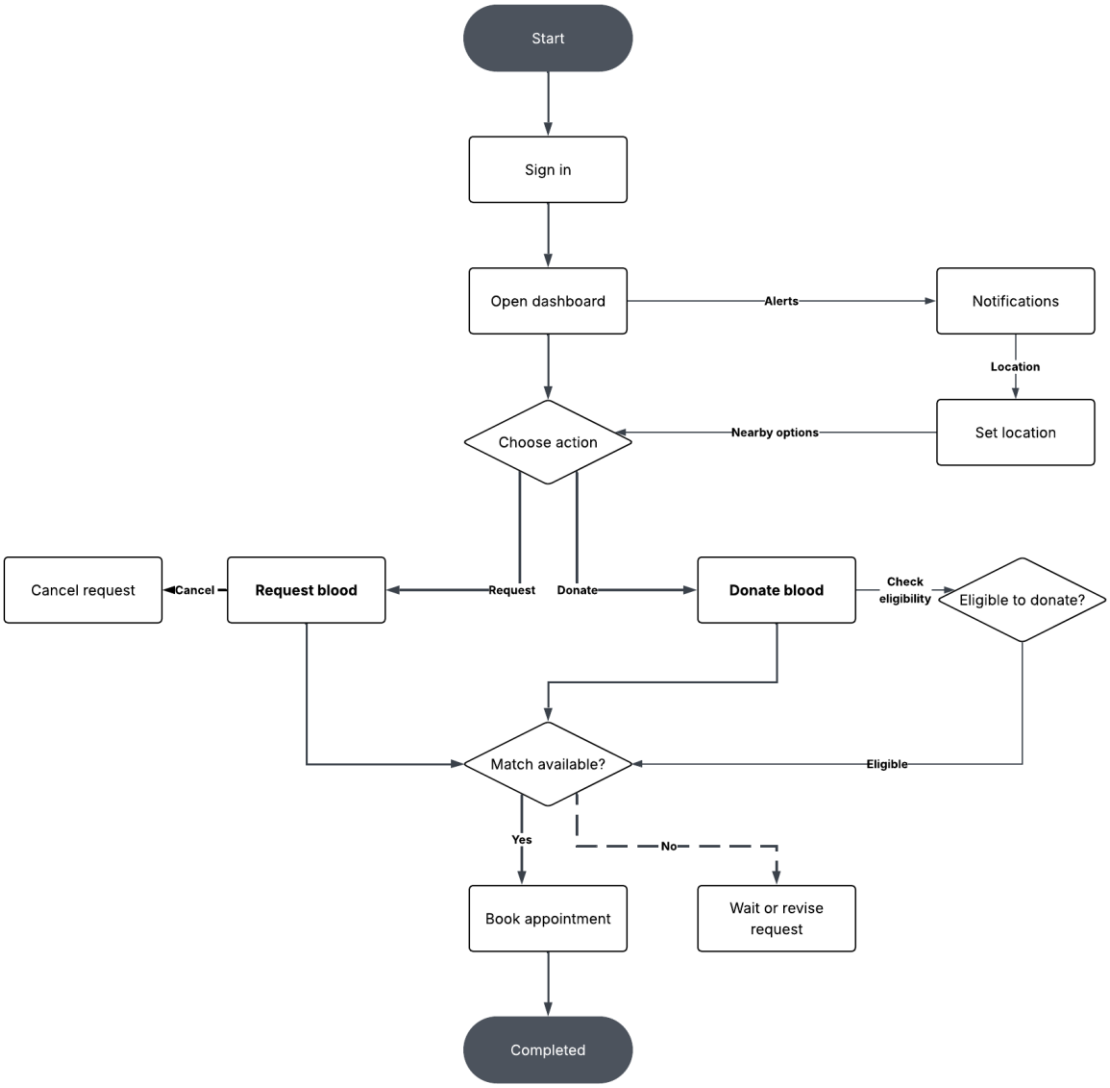


Figure 3.4: State Diagram of Bloodbridge

Sequence Diagram

The sequence Diagram of Bloodbridge helps organizations to track, store and manage blood donations for a profitable and efficient supply chain. It ensures that hospitals and healthcare centers can access the right type, amount and quality of blood on time.

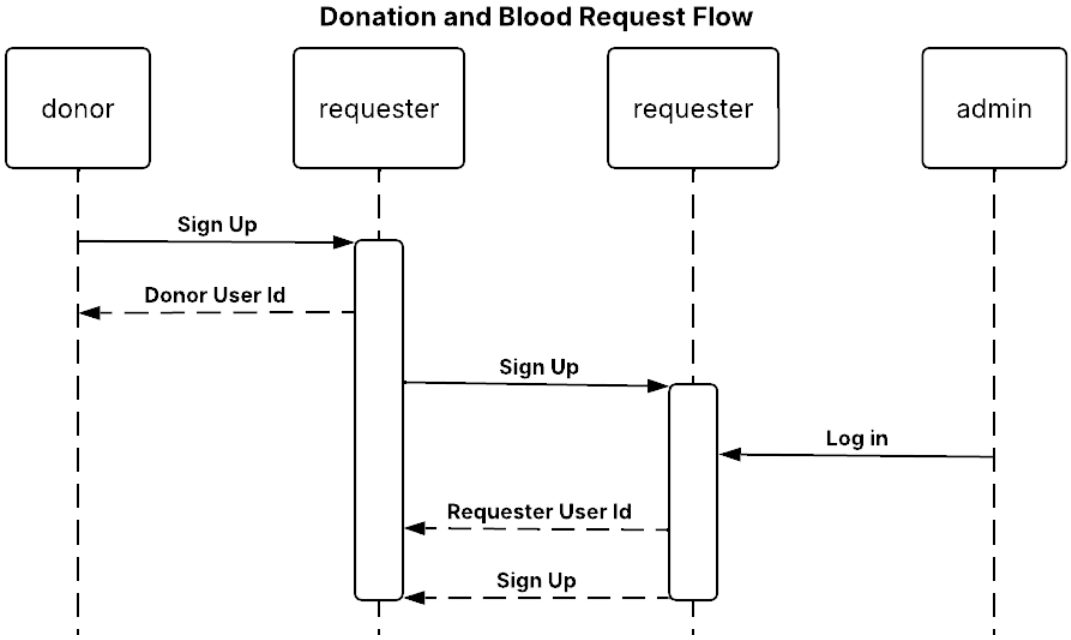


Figure 3.5: Sequence Diagram of Bloodbridge

3.1.5 Process Modeling using Activity Diagram

The activity diagram of Bloodbridge illustrates the two main methods of obtaining blood from a donor. The most frequent is simply taking the blood from a vein as whole blood. This blood is typically separated into parts, usually red blood cells and plasma since most recipients need only a specific component for transfusions. The activity diagram below shows that after login user can manage all the operations on Blood Cells, Stock, Blood, Donor, and Patient. These various objects interact throughout the Activity, and the user will not be able to access the page without first identifying their identity. As represented in activity diagram below, an activity diagram is a behavioral diagram that depicts the behavior of any system.

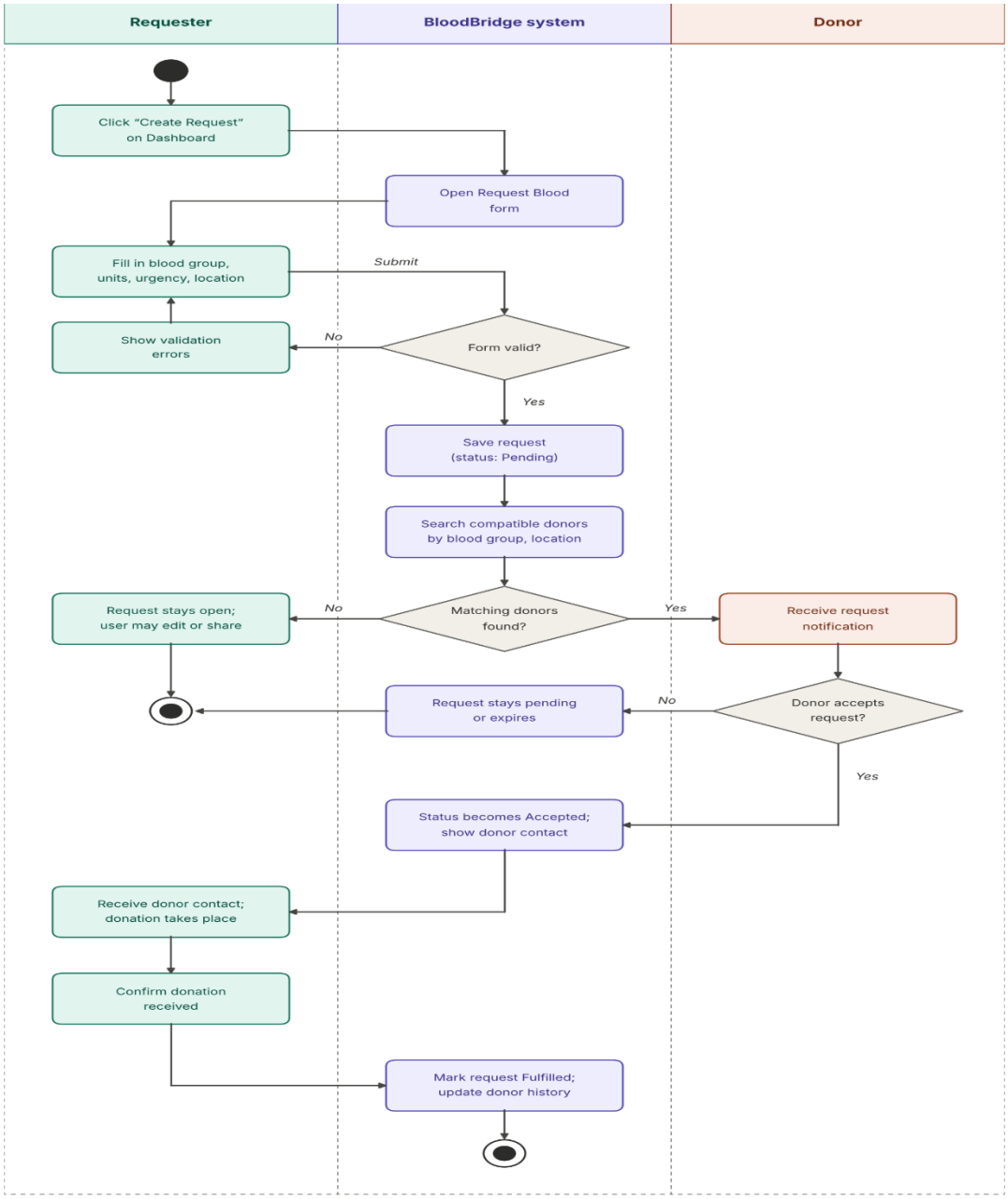


Figure 3.6 : Activity diagram of Bloodbridge

3.2 System Design

The system design phase translates the requirements and analysis into a detailed blueprint for development. It defines how system components interact, how data is structured, and how users will interface with the system.

3.2.1 Refinement of Class, Object, State, Sequence, and Activity Diagrams

The conceptual model of BloodBridge is refined into a clear design showing all core entities and their relationships. The default Laravel User model represents both regular users and administrators, with a role attribute separating the two. A single registered user can act as a donor, a requester, or both. Users can create a donor profile, post blood requests, submit donation offers, and receive notifications. The DonorProfile class stores donor-specific details such as blood group, city, weight, last donation date, and availability, and it is linked to exactly one user. The BloodRequest class is created by a user and records the patient's blood group, units needed, urgency, hospital, and a status that moves from Pending to Accepted and then to Fulfilled, or ends as Cancelled. The Donation class connects donors with blood requests: a donation may first exist as an open offer and is then linked to a request once the donor accepts it, and it tracks its own status from Offered to Matched and Completed. The Notification class is associated with the user and carries system messages, such as telling a requester that a donor has accepted. The administrator oversees the system by monitoring requests and donations and managing user accounts.

3.2.2 Component Diagram

The component diagram shows the system's modular structure. It consists of User Authentication Module, Donor Management Component, Blood Processing & Testing, Request & Matching Subsystem and a database component for storage. Each component communicates through defined interfaces, ensuring modularity, clear responsibilities, and maintainability.

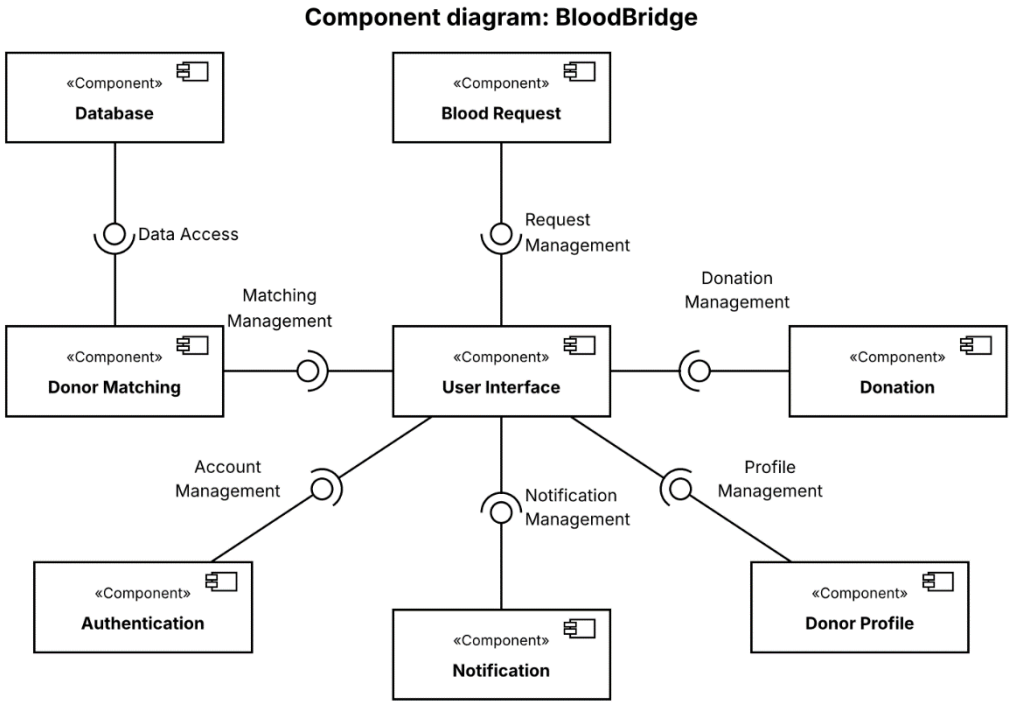


Figure 3.7: Component Diagram of bloodbridge

3.2.3 Deployment Diagram

A Deployment Diagram maps the physical execution architecture of the system. It shows how software artifacts (like containers, databases, and front-end builds) are deployed onto physical or virtual hardware nodes. A deployment diagram shows the physical side of the system: which machines (nodes) it runs on, which software runs on each machine, and how those machines communicate.

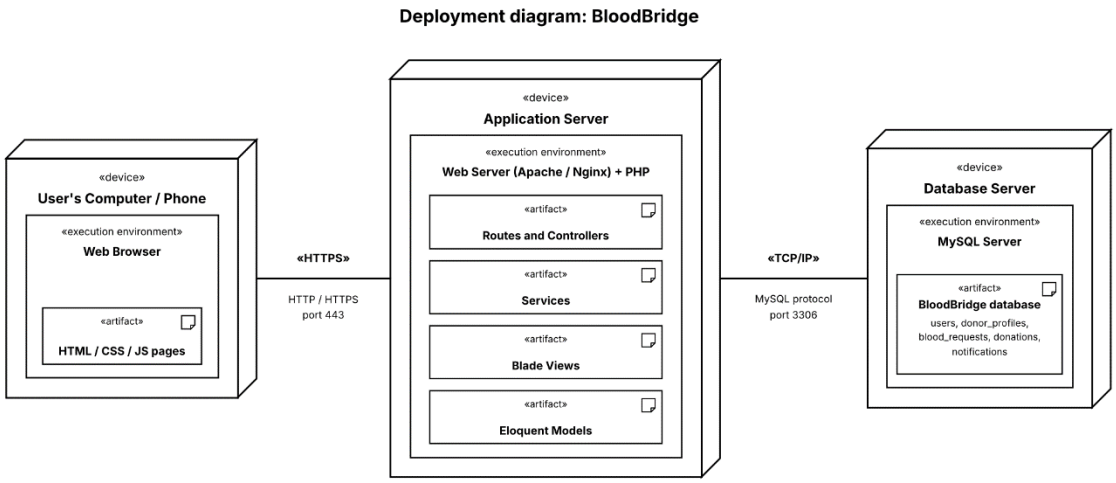


Figure 3.8: Deployment diagram of Bloodbridge

3.3 Algorithm Details

To improve the accuracy and relevance of donor search results, the system implements a weighted scoring algorithm that ranks potential donors based on multiple criteria. This logic is encapsulated in a Laravel service class called Donor Scoring Service.

The scoring system ensures that only eligible and relevant donors are suggested to recipients based on the following factors:

Key Scoring Criteria:

I. **Location Proximity (Weight: 35%):**
   Donors who are geographically closer to the request location receive higher scores. The Haversine formula is used to calculate distance based on latitude and longitude values.

II. **Eligibility (Weight: 25%):**
   Donors are considered eligible if at least 3 months have passed since their last donation. If not, they receive a score of 0 for this criterion.

III. **Blood Compatibility (Weight: 20%):**
   The donor's blood type is matched against a standard compatibility matrix. Incompatible donors are immediately filtered out.

IV. **Donor Regularity (Weight: 10%):**
   Based on the number of donations made in the last two years. Regular donors receive a proportionally higher score.

V. **Availability (Weight: 5%):**
   Donors can update their availability status. Only those marked as “available” receive full score here.

VI. **Health Status (Weight: 5%):**
   Self-declared health status (e.g., "good") is used to ensure the donor is fit for donation.

Additional Technical Highlights:

I. Caching: Laravel's Cache system is used to store intermediate results like calculated distances and donor activity, reducing repeated processing.
II. Filtering: After scoring, donors with zero blood compatibility or zero eligibility score are filtered out before the final sorting.
III. Sorting: The scored donors are sorted in descending order, ensuring the most suitable donors appear at the top of the result list.
IV. Validation and Logging: Input validation is performed to ensure correct data types, and Laravel's logging system records any scoring errors or missing data cases.

This algorithm provides an intelligent and efficient way to match recipients with the most suitable and currently available donors in their area.

Algorithm Steps:

- A user creates a blood request.
- The system reads the blood group, city and urgency.
- It collects all available donors.
- It removes donors with an incompatible blood group.
- It removes donors who are not eligible.
- It gives each donor a compatibility score. An exact match scores 1.0, and a compatible match scores 0.7.
- It gives each donor a location score. The same city scores 1.0, the same district 0.6, and anywhere else 0.2.
- It gives each donor a readiness score based on days since the last donation.
- It gives each donor a reliability score based on past responses.
- It multiplies each score by its weight.
- It adds the weighted scores to get a final score.
- It sorts the donors from highest score to lowest.
- It breaks ties by choosing the donor who donated longest ago.
- It notifies the top donors first.
- It notifies more donors if nobody accepts in time.
- It records each response to improve future scores.

CHAPTER 4. IMPLEMENTATION AND TESTING

This chapter describes how the Blood Donation Management System was built and tested. It outlines the tools used, key implementation details of the system modules, and the testing procedures carried out to ensure system functionality, reliability, and performance.

4.1 Implementation

The implementation phase involved developing the system components using Laravel PHP. The Laravel MVC structure enabled a clear separation between business logic, user interface, and data models. Blade templates were used to build user-friendly frontends, and Laravel's Eloquent ORM simplified database interactions.

The system includes several core modules, each responsible for different aspects of the platform, such as donor management, request handling, eligibility filtering, and admin operations. The Donor Scoring Service was implemented as a reusable service class, allowing efficient and scalable donor matching logic.

4.1.1 Tools Used

Below is a list of tools and technologies used during the development of the Blood Donation Management System:

I. **Laravel (PHP Framework):**
   Used as the core backend framework, offering powerful features like MVC structure, routing, Eloquent ORM, database migrations, authentication, and built-in security mechanisms.

II. **PHP (v8.x):**
   Server-side scripting language used with Laravel to implement business logic and system functionality.

III. **Visual Studio Code:**
   Code editor used during development, enhanced with Laravel and PHP extensions to improve code highlighting, IntelliSense, and debugging.

IV. **Cursor AI:**

An AI-powered coding assistant integrated into the development workflow to support code generation, completion, refactoring, and documentation within VS Code.

V. **MySQL / MariaDB:**
Relational database system used to store and manage donor information, donation records, and blood request data.

VI. **Laravel Artisan:**
Command-line tool included with Laravel, used to generate models, controllers, migrations, and run scheduled tasks.

VII. **Laravel Tinker:**
Interactive REPL tool used to test and debug Laravel Eloquent queries during development.

VIII. **Postman:**
Used to test and verify API routes and server responses, especially helpful for backend module validation.

IX. **Git:**
Version control system used for managing codebase changes and collaborating efficiently during development.

X. **Laravel Debugbar (optional):**
A developer toolbar used during testing and debugging to inspect database queries, routes, and performance metrics.

4.1.2 Implementation Details of Modules

The Blood Donation Management System was developed using the Laravel PHP framework following the MVC architecture. Below are the major modules implemented, along with a brief description of each:

1. User Registration and Authentication Module

- Handles the registration and login of both donors and recipients.
- Utilizes Laravel Breeze for secure authentication.
- Roles are assigned using middleware (donor, requester, admin).
- Password reset and email verification functionalities are included.

2. Donor Profile Management

- Donors can complete and update their profile including blood group, location (district/area), phone number, health condition, and last donation date.
- Automatically calculates donor eligibility (3 months rule) based on the last donation date.
- Donors can toggle availability status (Available / Not Available)

3. Donation Request System

- Recipients can submit requests by selecting the required blood group and location.
- Backend logic filters eligible donors and ranks them using the weighted scoring algorithm.
- Results are shown in a paginated, ranked donor list with contact options.

4. Admin Panel

- Built using Laravel Blade and custom dashboards.
- Admins can:
- Manually add/edit/delete donors.
- View all requests and donation records.
- Manage users and verify suspicious entries.
- Export donor or request data to Excel/CSV.

5. Donor History Tracking

- Every donation (whether confirmed via request or manually entered) is stored.
- Number of donations is tracked and visible on the donor's profile.
- Donation logs help in system analytics and monitoring donor activity.

6. Notification System (Optional / Future Integration)

- Designed for future integration using Laravel’s notification system.
- Notifications like SMS or email reminders can be triggered for:
- New requests matching donor criteria.
- Reminders for donors becoming eligible again.

7. Search and Filter Functionality

- Advanced filtering for both donors and requests based on:

a. Blood type

b. Location

c. Availability

- Search works in both donor dashboard and admin panel.

8. Validation and Security

- All user inputs are validated using Laravel's form request validation.
- Protected against common vulnerabilities like SQL injection, XSS, and CSRF.
- Route protection using auth and role middleware.

4.2 Testing

Testing plays a vital role in ensuring the quality and reliability of the Blood Donation Management System. Both unit testing and system testing were conducted throughout the development process using Laravel's built-in testing features and manual test scenarios.

4.2.1 Test Cases for Unit Testing

Unit testing focused on testing individual functions, controller methods, and logical conditions within the system. For instance:

Table 4.1: Unit testing of User and authentication Test in Bloodbridge

Project Name: Bloodbridge : Blood Donation Management system	
Test Case	
Test Case ID: TC-001	Test Designated Date: 24/04/2026
Test Case Type: Functional Testing	Test Executed By: Lakhmaya,Pemba Chhumu
Test Title: UserLogin	Test Execution Date: 29/04/2026
Test Data: username and password Summary: To verify login functionality of the user	

Steps	Test case	Input	Expected result	Status
1	Register with valid details	Name, unique email, password of 8 or more characters	User is saved and can log in	Pass
2	Register with a duplicate email	Email that already exists	Rejected with an "email already taken" error	Fail
3	Register with an empty name	Name left blank	Rejected with a validation error	Fail
4	Login with correct credentials	Valid email and password	Login succeeds and a session starts	Pass
5	Login with a wrong password	Valid email, wrong password	Login fails with an error message	Fail

Table 2 4.2: Unit testing of Donor Profile Test in Bloodbridge

Steps	Test case	Input	Expected result	Status
1	Eligible donor	Age 25, weight 60 kg, last donation 120 days ago	Is Eligible() returns true	Pass
2	Donor under minimum age	Age 17	Is Eligible() returns false	Fail
3	Donor who has never donated	No last donation date	Is Eligible() returns true	Pass
4	Invalid blood group	Blood group "Z+"	Rejected with a validation error	Fail
5	Change availability	Availability(true)	Is Available is saved	Pass

Table 3 4.3: Unit Testing of Blood Request Test in Blood bridge

Steps	Test case	Input	Expected result	Status
1	Create a valid request	B+, 2 units, city, future needed-by date	Request is saved with status Pending	Pass

2	Zero units	Units needed = 0	Rejected with a validation error	Fail
3	Cancel a pending request	cancel() on a Pending request	Status becomes Cancelled	Pass
4	Cancel a fulfilled request	cancel() on a Fulfilled request	Rejected, status stays Fulfilled	Fail
5	Valid status change	UpdateStatus(Accepted) on a Pending request	Status becomes Accepted	Pass
6	Edit a pending request	Change units from 2 to 3	Units are updated to 3	Pass

Table 4.4: Unit Testing of Donor Matching Test in Bloodbridge

Steps	Test case	Input	Expected result	Status
1	Exact group match	Request B+, donor B+, same city	Donor is in the results	Pass
2	Compatible group	Request B+, donor O+	Donor is in the results	Pass
3	Incompatible group	Request B+, donor A-	Donor is not in the results	Fail
4	Unavailable donor	Compatible donor with is Available false	Donor is not in the results	Fail
5	Available donor	Compatible donor with is Available true	Donor is in the results	Pass
6	No donors found	No compatible donors exist	Empty list is returned, no error	Fail

Each of these unit tests was verified to behave as expected, indicating that the individual components were functioning correctly.

4.2.2 Test Cases for System Testing

System testing evaluated the entire system as a whole, ensuring all modules interacted correctly and the platform behaved as expected in real-world scenarios.

Table 5 4.5:System Testing of Test Environment in BloodBridge

TEST-ID	Test Scenario	Steps	Expected Result	Status
ST-001	User Registration	1. User opens the Register page 2. User enters name, email and password 3. User submits the form	User account is created successfully	Pass
ST-002	User Login	1. User enters valid credentials 2. User clicks Login	User is redirected to the Dashboard	Pass
ST-003	Login with Wrong Password	1. User enters a valid email and a wrong password 2. User clicks Login	Login is refused and an error message is shown	Pass
ST-004	Create Donor Profile	1. User opens Donor Profile 2. User enters blood group, date of birth, weight and city 3. User saves the profile	Donor profile is saved and shown on the page	Pass
ST-005	Create Blood Request	1. User clicks Create Request 2. User enters blood group, units, city, urgency and needed-by date 3. User submits the request	Request is saved with status "Pending" and appears in My Requests	Pass
ST-006	Request with Missing Fields	1. User opens the Request Blood form 2. User leaves the blood group blank 3. User submits the form	Validation errors are shown and the request is not saved	Pass

ST-007	Donor Matching and Notification	1. A matching donor (same blood group, same city) is available 2. Requester submits a blood request	Matching donor receives a new request notification	Pass
ST-008	Incompatible Donor	1. The only available donor has an incompatible blood group 2. Requester submits a blood request	Donor is not notified and the request stays "Pending"	Pass
ST-009	Donate Blood Offer	1. Eligible donor clicks Donate Now 2. Donor enters date and location 3. Donor submits the offer	Donation offer is saved with status "Offered"	Pass
ST-010	Ineligible Donor	1. Donor who donated 20 days ago clicks Donate Now	A message explains that the donor is not yet eligible	Pass
ST-011	Accept a Request	1. Donor opens the request notification 2. Donor clicks Accept	Request status changes to "Accepted" and donation status to "Matched"	Pass
ST-012	Requester Notification	1. After a donor accepts, requester opens the Dashboard	Notification shows that the request was accepted, with the donor's contact details	Pass
ST-013	Complete Donation	1. Donor completes the donation and marks it complete 2. Requester confirms receipt	Request status is "Fulfilled", donation status is "Completed" and the donor's last donation date is updated	Pass
ST-014	Edit and Cancel Request	1. User opens My Requests	Units are updated, then	Pass

		2. User edits the units and saves 3. User clicks Cancel on the request	the status changes to "Cancelled"	
ST-015	Access Without Login	1. User logs out 2. User opens the Request Blood page by its address	User is redirected to the Login page	

All essential features and workflows were verified to work under normal and edge-case conditions, confirming the system's readiness for deployment.

4.3Result Analysis

The BloodBridge project demonstrates a well-structured and community-focused platform designed to connect blood donors with people who urgently need blood. The system combines donor and requester features in one environment, covering donor profiles, blood requests, donation offers, donor matching, and notifications.

BloodBridge provides a digital solution for connecting donors, requesters, and recipients. The platform streamlines the process of requesting blood, finding compatible donors, and tracking each donation from offer to completion. Its design promotes faster response times, better organisation, and stronger community participation in saving lives.

Functional Performance:

The project effectively supports key operational workflows, including:

- Registering users and managing donor profiles with blood group, city, and availability.
- Creating, editing, cancelling, and tracking blood requests.
- Submitting donation offers and accepting matching requests.
- Matching donors to requests by blood group, location, and eligibility.
- Sending notifications when a request is posted or accepted.
- Tracking request and donation status from Pending to Fulfilled.

These functions show that the platform is not only informational but also practical in supporting real-world blood donation efforts.

Analytical Value:

The result analysis measures platform performance through indicators such as:

- Request fulfilment rate (fulfilled requests ÷ total requests).
- Donor response rate (requests accepted ÷ donors notified).
- Average time to match (from request creation to acceptance).
- Donor eligibility rate (eligible donors ÷ registered donors).
- Number of pending requests awaiting a donor.
- Donation and request activity over time.
- Demand and supply by blood group.

Social and Operational Impact:

The project shows strong potential for social impact and sustainability. By reducing the time needed to find a compatible donor, the system can improve response in emergencies. Status tracking and notifications keep requesters and donors informed, and the structure makes the system suitable for scaling and long-term management.

CHAPTER 5. CONCLUSION AND FUTURE RECOMMENDATIONS

5.1 Conclusion

The Blood Donation Management System successfully provides a digital platform to connect blood donors with recipients in need, improving accessibility and efficiency in blood donation processes. The system allows donors to register, track their donations, and update their availability, while recipients can easily find eligible donors based on location and blood group. The admin panel facilitates effective management of donor data and requests.

By implementing a weighted scoring algorithm, the system ensures that the most suitable donors are prioritised, improving the likelihood of timely donations. The use of the Laravel PHP framework and modern web development practices contributed to a robust, secure, and maintainable solution.

5.2 Lessons Learned / Outcome

- Importance of clear requirement analysis and iterative design to meet user needs.
- The value of modular architecture (MVC) in simplifying development and maintenance.
- Leveraging Laravel's built-in features like authentication, ORM, and caching significantly accelerates development.
- The significance of rigorous testing to ensure system reliability and security.
- Practical experience in integrating complex algorithms like donor scoring in a real-world application.

5.3 Future Recommendations

To further enhance the system, the following features and improvements are recommended:

- Integration of SMS or email notifications to alert donors of new requests and remind them when they become eligible again.

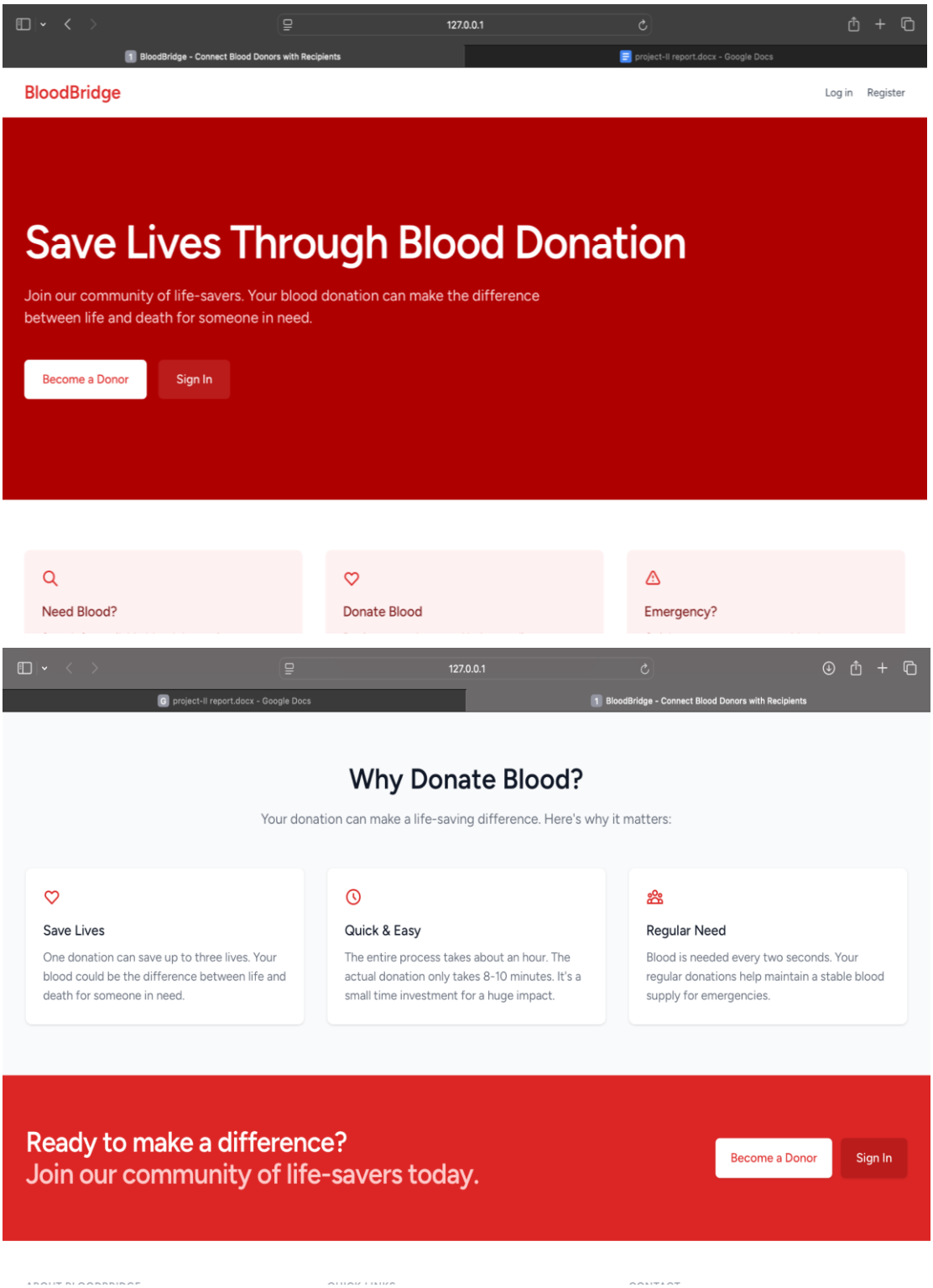
- Mobile app development for easier access and better user engagement.
- Implementation of real-time chat or communication tools between donors and recipients.
- Enhanced analytics dashboard for admins to monitor blood donation trends and manage resources effectively.
- Support for multiple languages to cater to a broader user base.
- Implementation of a more detailed health check reporting system for donors.
- Integration with national or regional blood bank databases for wider coverage and data sharing.

References

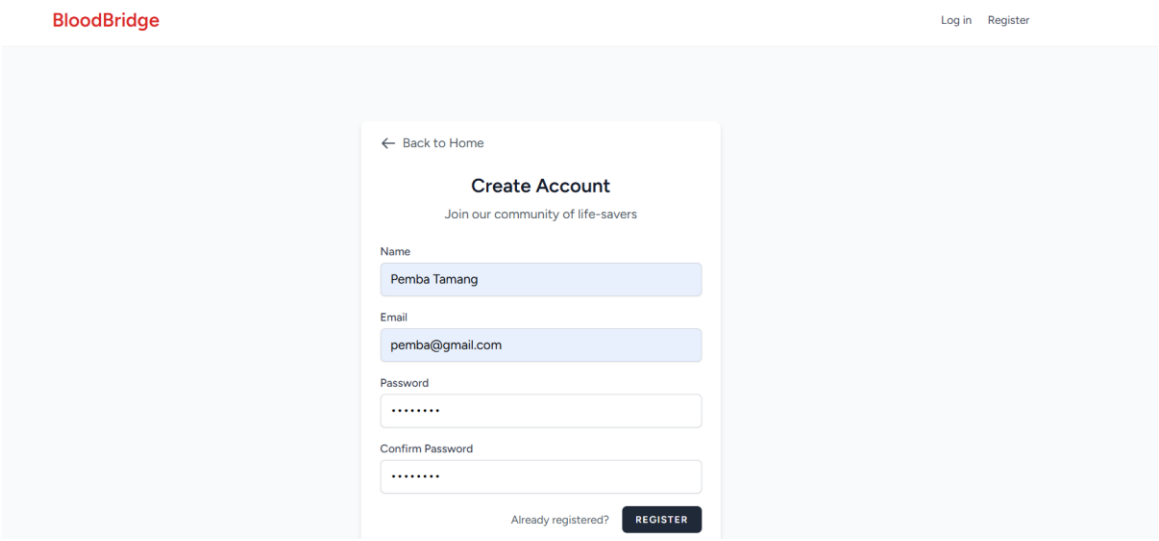
- Laravel Documentation. Laravel Framework. Available at: https://laravel.com/docs
- PHP Official Website: PHP: Hypertext Preprocessor. Available at: https://www.php.net
- Blood Donation Guidelines, World Health Organisation. Blood Donor Selection. Available at: https://www.who.int/publications/i/item/blood-donor-selection-guidelines
- Geographical Distance Calculation: Haversine Formula for distance between two coordinates. Available at: https://www.movable-type.co.uk/scripts/latlong.html
- Laravel Eloquent ORM Laravel ORM documentation. Available at: https://laravel.com/docs/eloquent
- Web Accessibility Guidelines W3C Web Accessibility Initiative (WAI). Available at: https://www.w3.org/WAI/standards-guidelines

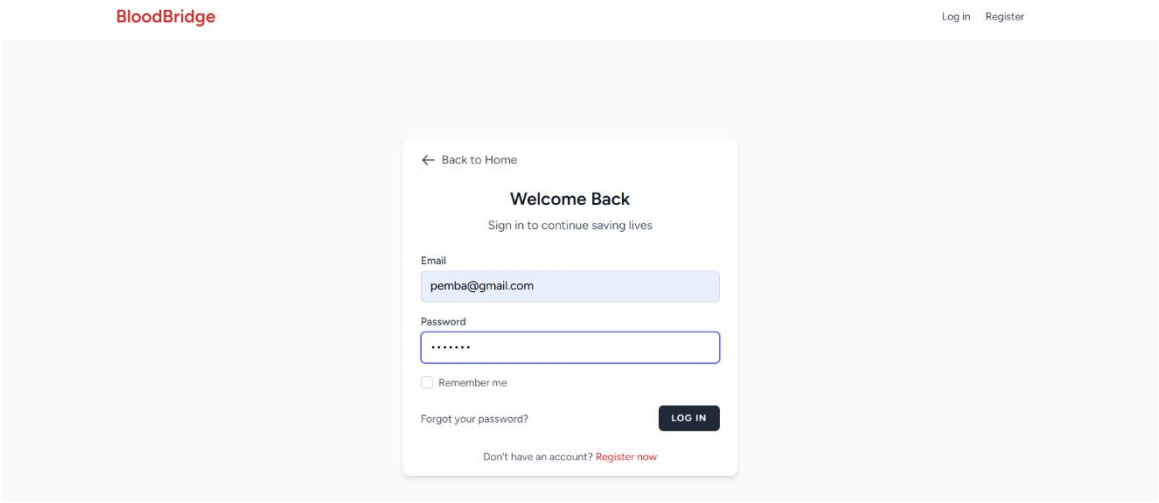
Chapter 6. Appendices

- Homepage

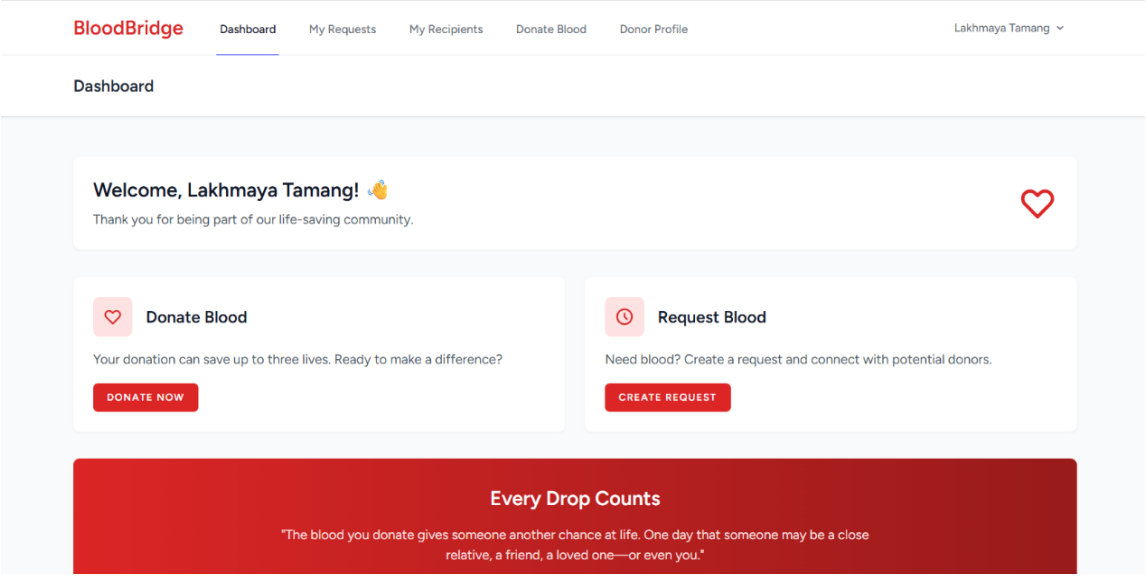


User Authentication Page: Screenshot showing user authentication form with input validation.

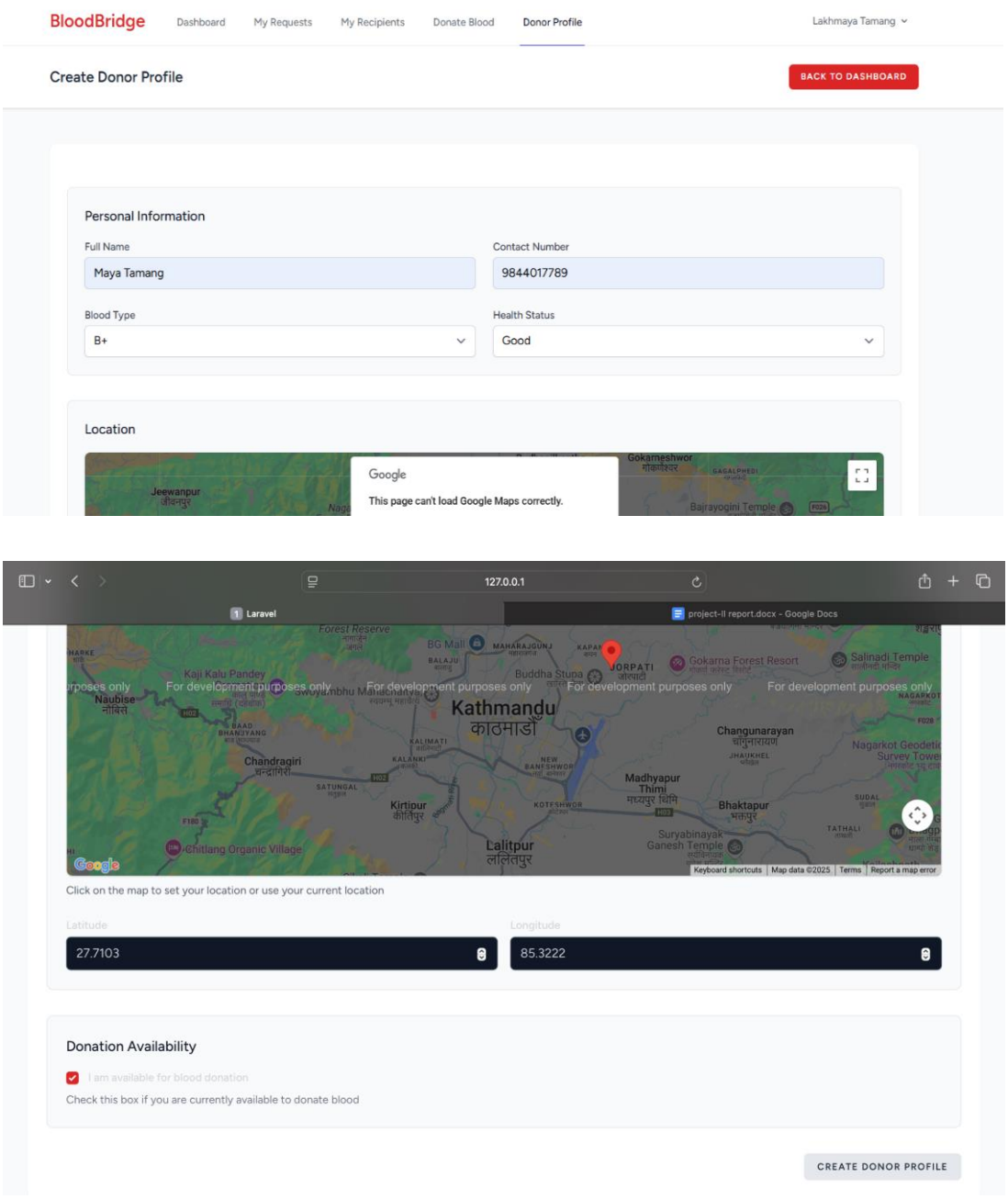




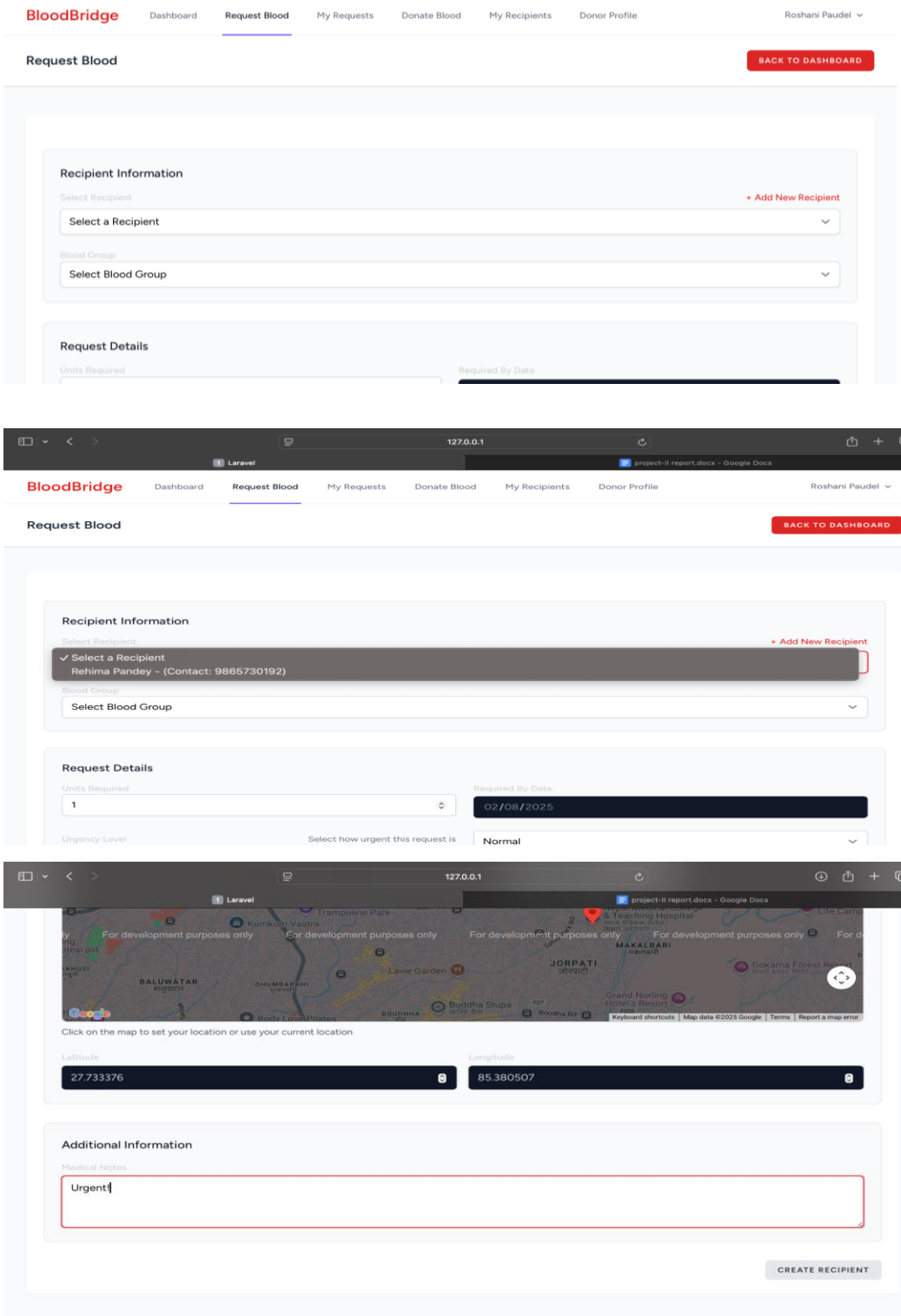
User Dashboard Page: Screenshot showing the user dashboard page.



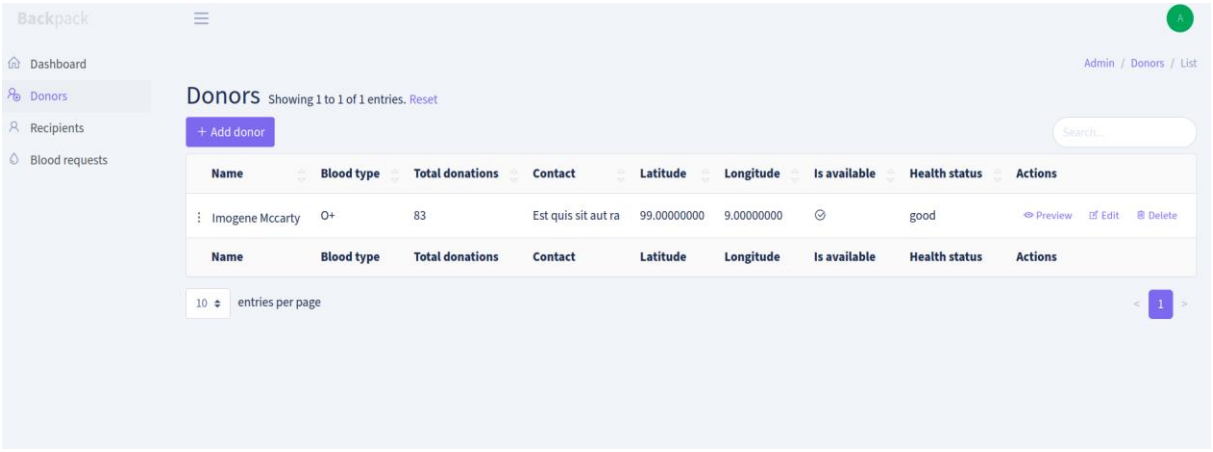
Blood Donation Page: Screenshot showing the blood donation page.

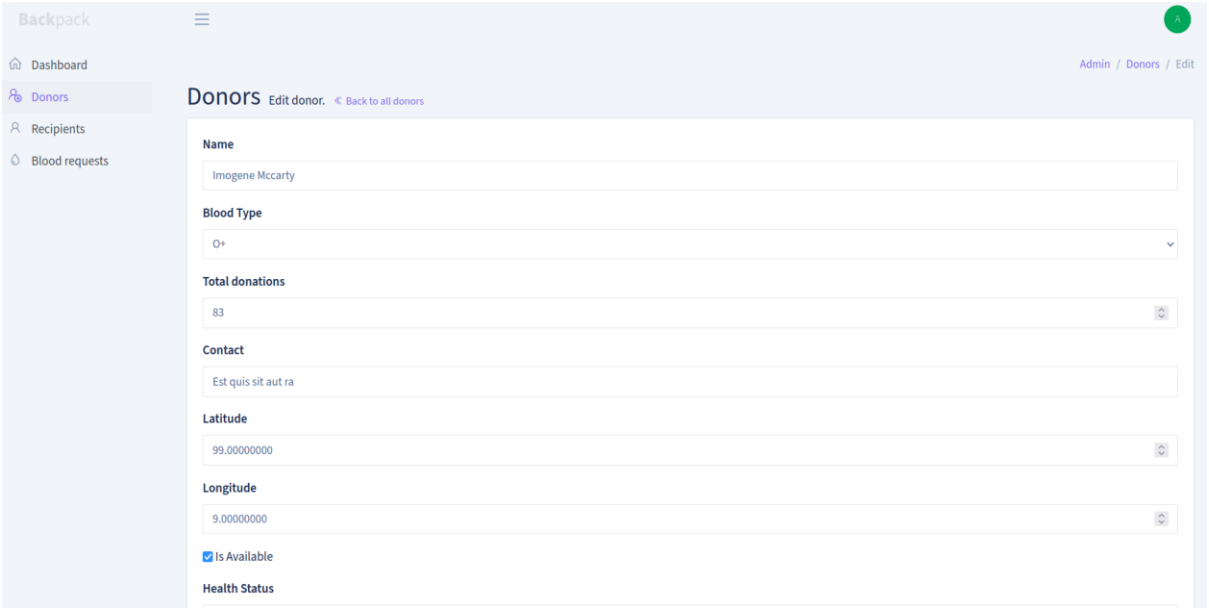


Blood Request Page: Screenshot showing the blood Request page.



Admin Donor CRUD





Admin Recipients CRUD

